

7.15 补题报告

2026 山东省大学生程序设计竞赛 (SDCPC) · 25 李增睿 · 补题 10 / 13 (除 B, F, M)

- 拓扑排序 + bitset
- 数位 DP
- 线段树 DP
- 贪心 / 构造
- Kruskal
- 数论

题号	题目	算法	难度
A	华容道	区间连边 拓扑排序 bitset 传递闭包	难
C	会议日程	整数三分 凸函数	中
D	最大数码 2	数位 DP 双上界 进位处理 二维差分	难
E	简单的构造题	构造 棋盘染色	中
G	吸血鬼爬行者	贪心 排序	中
H	拼图	DFS 爆搜 剪枝 计数	中
I	版本号	签到	签到
J	插松枝	DP 线段树 值域分层 边界规约	最难
K	最小生成树	Kruskal 贪心 分类讨论	中
L	分数迭代	数论 gcd 断点预处理	难

Problem A. 华容道

A. 华容道

区间连边

拓扑排序

bitset 传递闭包

难

复杂度 $O(4NM + (NM)^2/w)$, $NM \leq 5 \times 10^4$

每个格子有一个滑动方向, 求每个格子出发能经过多少格子, 走不出循环的输出 -1. 本质是有向图可达性计数.

关键是**建边不能暴力**: 直接对滑动路径上每个格子连边是 $O((NM)^2)$ 的. 观察到一行内两个相邻的 **L** 格子之间的区段是"共享"的 -- 对每个方向按行/列扫一遍, 每个 **L** 格向 [上一个 L 的位置, 自身) 的区段连边, 四个方向同理, 总边数摊到 $O(4NM)$.

建好图后求拓扑序: 在环上的格子(进不了拓扑序)答案为 -1; 其余格子按拓扑序用

`bitset<50001>` 逆序合并可达集合, `dp[u].count()` 即为答案. 这是数据范围 $N \leq 5 \times 10^4$ 时传递闭包计数的标准套路.

A.cpp

```
// @Author : GoryK
#include <bits/stdc++.h>
using namespace std;

#define int int64_t

using i32 = int32_t;
using i64 = int64_t;

#ifdef LOCAL
#include "__DEBUG_TOOL.h"
#else
#define debug(...)
#endif

void solve() {
    int n, m;
    cin >> n >> m;
    int N = n * m;

    auto dc = [&](char ch) -> int {
        if (ch == 'U') return 0;
        if (ch == 'L') return 1;
        if (ch == 'D') return 2;
        if (ch == 'R') return 3;
        return -1;
    };
}
```

```
};

auto id = [&](int x, int y) → int {
    return (x - 1) * m + y;
};

vector<vector<int>> g(n + 1, vector<int>(m + 1));
for (int i = 1; i ≤ n; i++)
    for (int j = 1; j ≤ m; j++) {
        char ch; cin >> ch;
        g[i][j] = dc(ch);
    }

vector<vector<int>> G(N + 1);
for (int i = 1; i ≤ n; i++) {
    int lst = -1;
    for (int j = 1; j ≤ m; j++) {
        if (g[i][j] == 1) {
            for (int k = (lst == -1 ? 1 : lst); k < j; k++)
                G[id(i, j)].push_back(id(i, k));
            lst = j;
        }
    }
}

for (int i = 1; i ≤ n; i++) {
    int lst = -1;
    for (int j = m; j ≥ 1; j--) {
        if (g[i][j] == 3) {
            for (int k = (lst == -1 ? m : lst); k > j; k--)
                G[id(i, j)].push_back(id(i, k));
            lst = j;
        }
    }
}

for (int j = 1; j ≤ m; j++) {
    int lst = -1;
    for (int i = 1; i ≤ n; i++) {
        if (g[i][j] == 0) {
            for (int k = (lst == -1 ? 1 : lst); k < i; k++)
                G[id(i, j)].push_back(id(k, j));
            lst = i;
        }
    }
}

for (int j = 1; j ≤ m; j++) {
    int lst = -1;
    for (int i = n; i ≥ 1; i--) {
```

```
        if (g[i][j] == 2) {
            for (int k = (lst == -1 ? n : lst); k > i; k --)
                G[id(i, j)].push_back(id(k, j));
            lst = i;
        }
    }
}

vector<vector<int>> rev(N + 1);
for (int u = 1; u ≤ N; u++) {
    for (int v : G[u]) {
        rev[v].push_back(u);
    }
}

vector<int> indg(N + 1);
for (int i = 1; i ≤ N; i++) {
    indg[i] = G[i].size();
}

queue<int> q;
for (int i = 1; i ≤ N; i++) {
    if (!indg[i]) {
        q.push(i);
    }
}

vector<int> topo, vis(N + 1);
while (!q.empty()) {
    int v = q.front(); q.pop();
    topo.push_back(v);
    vis[v] = 1;
    for (int u : rev[v]) {
        if (--indg[u] == 0) {
            q.push(u);
        }
    }
}

// Bitset DP (逆序)
vector<bitset<50001>> dp(N + 1);
for (int u : topo) {
    dp[u].set(u);
    for (int v : G[u]) {
        dp[u].set(v);
        dp[u] |= dp[v];
    }
}

for (int i = 1; i ≤ n; i++) {
```

```
for (int j = 1; j ≤ m; j++) {
    int idx = id(i, j);
    if (vis[idx])
        cout << dp[idx].count() << " ";
    else
        cout << -1 << " ";
    if (j == m) cout << "\n";
}
}
}

int32_t main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int tt = 1, _ = 0;
    std::cin >> tt;
    while (tt-- and ++_)
        // std::cout << "TEST CASE : " << _ << endl,
        solve();

    return 0;
}
```

Problem C. 会议日程

C. 会议日程

整数三分

凸函数

中

复杂度 $O(n \log V)$

把代价写成 $f(x) = \sum \min((l_i - x)^2, (r_i - x)^2)$ (x 落在区间内贡献 0). 每一项都是关于 x 的凸函数, 凸函数之和仍是凸(单谷)函数, 直接整数三分.

补题时顺手总结了三分的本质: 每次取 m_1, m_2 两个点, 求最小值就舍弃函数值更大的一侧. 之所以两个点就够, 是因为单谷函数只有一个"折返点", 只需要判断当前在折返点左侧还是右侧 -- 两个点的相对大小恰好给出这个信息.

C.cpp

```
// @Author : GoryK
#include <bits/stdc++.h>
using namespace std;

#define int int64_t

#ifdef LOCAL
#include "__DEBUG_TOOL.h"
#else
#define debug(...)
#endif

// REGISTER_REFLECT()

void solve() {
    int n;
    cin >> n;
    using pii = pair<int, int>;
    vector<pii> a(n + 1); int lim = 0;
    for (int i = 1; i <= n; i++) {
        auto &[l, r] = a[i];
        cin >> l >> r;
        lim = max({lim, l, r});
    }

    // 三分就是每次挑两个端点进行分析
    // 如果要求最大的, 就每次舍弃更小的一侧
    // 求最小就舍弃另外一侧
    // 原因不是点数不够, 而是问题本身变难了。单峰函数优美的地方在于:
    // 函数只有一个"折返点", 所以你只需要判断当前处于折返点的左侧还是右侧 — 两个点的
    // 相对大小就够了。
```

```
auto f = [&](int x) → int {
    int res = 0;
    for (int i = 1; i ≤ n; i++) {
        auto [l, r] = a[i];
        if (l ≤ x and x ≤ r)
            continue;
        res += min(abs(l - x) * abs(l - x), abs(r - x) * abs(r - x));
    }

    return res;
};

int l = 0, r = lim, ans = l;
while (l ≤ r) {
    int m1 = l + (r - l) / 3;
    int m2 = r - (r - l) / 3;
    if (f(m1) > f(m2)) {
        l = m1 + 1, ans = m2;
    } else {
        r = m2 - 1, ans = m1;
    }
}

cout << f(ans) << "\n";
}

int32_t main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int tt = 1, _ = 0;
    std::cin >> tt;
    while (tt-- and ++_)
        // std::cout << "TEST CASE : " << _ << endl,
        solve();

    return 0;
}
```

Problem D. 最大数码 2

D. 最大数码 2

数位 DP

双上界

进位处理

二维差分

难

复杂度 $O(9 \times \text{len} \times 4 \times 12 \times 2 \times 200)$

求 $\sum \text{maxdigit}(a+b)$, $a \in [la, ra]$, $b \in [lb, rb]$. 枚举最大数码 $mx_d \in [1, 9]$, 统计 $a+b$ 的最大数码恰为 mx_d 的对数(所有位 $\leq mx_d$ 且至少一位 $= mx_d$), 答案累加 $mx_d \times$ 对数. 二维区间用 4 项前缀容斥拆开.

难点在于 a 和 b 要**同时**数位 DP, 且和的数位受进位影响. 处理方式: 从高位往低位 DFS, 状态里带上"本位需要向高位进的位 $carry$ "; 转移时枚举本位数字 da, db 和**低位送上来**的进位 cin , 校验 $(da+db+cin \geq 10) == carry$, 再把 cin 作为子问题的 $carry$ 传下去 -- 相当于把"进位链"拆成逐位的自治约束.

状态: $dp[\text{位}][a \text{ 顶界}][b \text{ 顶界}][mx_d \text{ 出现次数}][carry]$, 每个 mx_d 重置记忆化.

D.cpp

```
// @Author : GoryK
#include <bits/stdc++.h>
using namespace std;

#define int int64_t
#define fir first
#define sec second

#define ckmn(a, b) ((b) < (a) ? (a) = (b), 1 : 0)
#define ckmx(a, b) ((b) > (a) ? (a) = (b), 1 : 0)

#ifdef LOCAL
#include "__DEBUG_TOOL.h"
#else
#define debug(...)
#endif

// REGISTER_REFLECT()

void solve() {
    int la, ra, lb, rb;
    cin >> la >> ra >> lb >> rb;

    using pbb = pair<bool, bool>;
    static int dp[12][2][2][12][2];
    // tight, cnt_mx
```



```

int mx_d = 1; string A, B;
// carry 是这一位是否要求产生进位
auto dfs = [&](this auto&& dfs, int P, pbb T, int cnt_mx, int carry) → int
{
    int len = A.size();
    if (P == len) {
        return (carry == 0 && cnt_mx > 0);
    }

    auto [Ta, Tb] = T;
    if (dp[P][Ta][Tb][cnt_mx][carry] ≠ -1) {
        return dp[P][Ta][Tb][cnt_mx][carry];
    }

    int res = 0;
    int lim_a = T.fir ? A[P] - '0' : 9LL;
    int lim_b = T.sec ? B[P] - '0' : 9LL;

    for (int da = 0; da ≤ lim_a; da++) {
        for (int db = 0; db ≤ lim_b; db++) {
            // 枚举低位(P + 1)向本位的进位, 这个没有枚举
            for (int cin = 0; cin ≤ 1; cin++) {
                int dsum = da + db + cin, dcur = dsum % 10;
                if (dcur > mx_d) continue;
                // 本位向高位的进位必须与父要求一致
                if ((dsum ≥ 10) ≠ carry) continue;
                res +=
                    dfs(
                        P + 1,
                        pbb(T.fir & (da == (A[P] - '0')), T.sec & (db == (B[P] - '0'))),
                        cnt_mx + (dcur == mx_d),
                        cin // 子调用的 carry = 本位收到的低位进位
                    );
            }
        }
    }

    return dp[P][Ta][Tb][cnt_mx][carry] = res;
};

auto chg = [&](int a, int b) → void {
    A = to_string(a), B = to_string(b);
    int lim = max(A.size(), B.size()) + 1;
    while (A.size() < lim) A = '0' + A;
    while (B.size() < lim) B = '0' + B;
    memset(dp, -1, sizeof dp);
};

int ans = 0;
while (mx_d ≤ 9) {

```

```
    chg(ra, rb);
    ans += mx_d * dfs(0, pbb(true, true), 0, false);

    chg(la - 1, rb);
    ans -= mx_d * dfs(0, pbb(true, true), 0, false);

    chg(ra, lb - 1);
    ans -= mx_d * dfs(0, pbb(true, true), 0, false);

    chg(la - 1, lb - 1);
    ans += mx_d * dfs(0, pbb(true, true), 0, false);

    // debug(mx_d);
    mx_d ++;
}

cout << ans << "\n";
}

int32_t main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int tt = 1, _ = 0;
    std::cin >> tt;
    while (tt-- and ++_)
        // std::cout << "TEST CASE : " << _ << endl,
        solve();

    return 0;
}

// // 这个题 其实可以去掉进位的影响
// // 每一位最多进位是 1,
// // 如果 我的最大数字不是 9 的话就一定不会产生连环进位
// // 如果 mx_d = 9 了, 那么进位之后最多也只会产生 0, 不会威胁到 mx_d 的实现
// // 哦但是这样有可能影响到 mx_d 的出现次数
```

Problem E. 简单的构造题

E. 简单的构造题

构造

棋盘染色

中

复杂度 $O(nm)$

构造 $n \times m$ 网格填 $0 \sim nm-1$, 使相邻格子的数值不相邻. 按 $(i+j)$ 奇偶做棋盘染色: 先给偶类格子连续编号 $0, 1, 2, \dots$, 再给奇类编号. 相邻格子必属不同色类, 两类编号至少相差约 $nm/2 \geq 2$, 自然满足条件.

特判: $nm \leq 4$ 无解; 2×3 时偶类最大编号(2)与奇类最小编号(3)恰好相邻会冲突, 手动给出一组解.

E.cpp

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
#define ll long long

void solve() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> g(n + 1, vector<int>(m + 1));

    if (n * m <= 4) {
        cout << "No" << "\n";
        return;
    }

    if (n == 2 and m == 3) {
        cout << "Yes" << "\n";
        cout << "5 3 1\n";
        cout << "2 0 4\n";
        return;
    }

    int num = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (!((i + j) & 1)) {
                g[i][j] = num++;
            }
        }
    }
}
```

```
for (int i = 1; i ≤ n; i++) {
    for (int j = 1; j ≤ m; j++) {
        if ((i + j) & 1) {
            g[i][j] = num++;
        }
    }
}

cout << "Yes" << "\n";
for (int i = 1; i ≤ n; i++) {
    for (int j = 1; j ≤ m; j++) {
        cout << g[i][j] << " \n"[j == m];
    }
}

int32_t main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

    int tt = 1;
    cin >> tt;
    while (tt --) {
        solve();
    }

    return 0;
}
```

Problem G. 吸血鬼爬行者

G. 吸血鬼爬行者

贪心

排序

中

复杂度 $O(n \log n)$

按颜色分组, 每组内力量降序排序. 贪心结论: 第 j 轮从每种颜色各取出其第 j 大的一张组成一层, 同一层内沿颜色值扫描, 颜色连续 ($c == pc+1$ 且上一张非空) 时连击计数 cnt 递增, 该张贡献 $cnt \times val$.

直觉: 连击系数只和"颜色连续段的长度"有关, 让每种颜色最大的牌们互相组队能把最大的系数乘到最大的权值上(排序不等式), 逐层取第 j 大即可.

G.cpp

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
#define ll long long

void solve() {
    int n; cin >> n;
    map<int, vector<int>>> mp;
    int mx = 0;
    for (int i = 1; i <= n; i++) {
        int c, d; cin >> c >> d;
        mp[c].push_back(d);
        mx = max(mx, (int)mp[c].size());
    }

    for (auto &[c, v] : mp) sort(v.rbegin(), v.rend());

    int ans = 0;
    for (int j = 0; j < mx; j++) {
        int cnt = 0, pc = 0, pv = 0;
        for (auto &[c, v] : mp) {
            int val = j < (int)v.size() ? v[j] : 0;
            cnt = (c == pc + 1 && pv) ? cnt + 1 : 1;
            ans += cnt * val;
            pc = c, pv = val;
        }
    }
    cout << ans << "\n";
}
```

```
int32_t main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

    int tt = 1;
    cin >> tt;
    while (tt --) {
        solve();
    }

    return 0;
}
```

Problem H. 拼图

H. 拼图

DFS 爆搜

剪枝

计数

中

复杂度 $O(\text{方案树规模})$, 靠定位剪枝

统计用若干碎片(不可旋转)恰好铺满棋盘空位的方案数. 核心剪枝: 每次找**行优先第一个空格**, 它必须被某个剩余碎片的**第一个 '#'** (同样行优先)覆盖 -- 对齐方式因此唯一, 每个碎片只需尝试一种放法.

这个"最小空格锚点"技巧保证同一方案只会以一种顺序被搜到一次, 不重不漏; 越界或与已有方块重叠即剪枝, 铺满时计数 +1.

H.cpp

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
#define ll long long

void solve() {
    int n, m, K;
    cin >> n >> m >> K;
    vector<vector<int>>> grid(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            char c;
            cin >> c;
            grid[i][j] = (c == '#');
        }
    }

    // 1 代表有, 0 代表没有
    vector<vector<vector<int>>>> pins(K + 1);
    for (int i = 1; i <= K; i++) {
        int r, c;
        cin >> r >> c;
        pins[i].resize(r);
        for (int j = 0; j < r; j++) {
            for (int k = 0; k < c; k++) {
                char c; cin >> c;
                pins[i][j].push_back(c == '#');
            }
        }
    }
}
```

```
map<int, array<int, 2>> corner;
for (int i = 1; i ≤ K; i++) {
    bool f = false;
    for (int j = 0; j < pins[i].size(); j++) {
        for (int k = 0; k < pins[i][j].size(); k++) {
            if (pins[i][j][k]) {
                corner[i] = {j, k};
                f = true;
                if (f) break;
            }
        }
        if (f) break;
    }
}

vector<int> REM;
for (int i = 1; i ≤ K; i++) REM.push_back(i);

auto chk_yes = [&](const auto &g) → bool {
    for (int i = 1; i ≤ n; i++) {
        for (int j = 1; j ≤ m; j++) {
            if (g[i][j] == 0) {
                return false;
            }
        }
    }
    return true;
};

auto find_corner = [&](const auto &g) → array<int, 2> {
    for (int i = 1; i ≤ n; i++) {
        for (int j = 1; j ≤ m; j++) {
            if (g[i][j] == 0) {
                return {i, j};
            }
        }
    }
    return {-1, -1};
};

auto chk_coor = [&](int x, int y) → bool {
    return x > 0 and x ≤ n and y > 0 and y ≤ m;
};

int ans = 0;
auto dfs = [&](auto &&sef, vector<vector<int>>& G, vector<int>& rem) → void {
    if (chk_yes(G)) {
        ans++;
    }
}
```



```
        return;
    }

    auto [X, Y] = find_corner(G);
    for (int &idx : rem) {
        if (idx == 0) continue;
        auto [x, y] = corner[idx];
        int difx = x - X, dify = y - Y;
        bool f = true;
        for (int i = 0; i < pins[idx].size(); i++) {
            for (int j = 0; j < pins[idx][i].size(); j++) {
                int curx = i - difx, cury = j - dify;
                if (!chk_coor(curx, cury) or pins[idx][i][j] & G[curx][cury]) {
                    f = false;
                    break;
                }
            }
        }
        if (!f) break;
    }

    if (!f) continue;
    auto g = G;
    for (int i = 0; i < pins[idx].size(); i++) {
        for (int j = 0; j < pins[idx][i].size(); j++) {
            int curx = i - difx, cury = j - dify;
            if (pins[idx][i][j]) {
                g[curx][cury] = 1;
            }
        }
    }

    int temp = idx; idx = 0;
    sef(sef, g, rem);
    idx = temp;
}

dfs(dfs, grid, REM);

cout << ans << "\n";
}

int32_t main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

    int tt = 1, _ = 0;
    cin >> tt;
    while (tt -- and ++_) {
```

```
// cout << "TEST_CASE: " << _ << "\n";  
solve();  
}  
  
return 0;  
}
```

Problem I. 版本号

I. 版本号

签到

签到

复杂度 $O(n \log n)$

把 (主版本, 次版本) 作为 pair 排序, 取最大的输出即可.

I.cpp

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
#define ll long long

using pii = pair<int, int>;

void solve() {
    int n; cin >> n;
    vector<pii> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i].first >> a[i].second;
    }

    sort(a.begin(), a.end(), greater<pii>());
    cout << a[0].first << ' ' << a[0].second << "\n";
}

int32_t main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

    int tt = 1;
    cin >> tt;
    while (tt--) {
        solve();
    }

    return 0;
}
```

Problem J. 插松枝

J. 插松枝

DP

线段树

值域分层

边界规约

最难

复杂度 $O(n \log n)$

按值从大到小分批删除, 每批删完后隔板 (divider) 只可能停在本批区间的左端或右端, 于是设 $dp[i][0/1]$ = 处理完第 i 大的值后, 隔板停在 $L-1$ 缝 / R 缝的最小移动次数, 答案为 $n + \min(dp)$.

这题真正的坑在**边界口径**, 补题时专门做了统一规约: 用整数 b 表示隔板卡在位置 b 与 $b+1$ 之间的"缝", 停在 R 即 R 的右侧缝 ($=cr$), 停在 L 即 L 的左侧缝 ($=cl-1$).

- **缝到缝移动 $T(lo, hi)$** : 跨过的位置是半开半闭 $[lo, hi]$, 代价 = $(hi-lo) - \text{已删除数} - \text{本批待删数}$, 三个量全用 $[lo+1, hi]$ 口径;
- **整段扫描 sweep**: 覆盖闭区间 $[L, R]$, 代价 = $(cr-cl+1) - \text{已删除数} - k$, 全用 $[cl, cr]$ 口径;
- 已删除的元素数量用**线段树**单点加 + 区间查询维护, 本批元素在有序位置数组上二分统计.

两段衔接在位置上首尾相接: T 覆盖到缝 $cl-1$, $sweep$ 从位置 cl 开始, 不缺格不重复; 本批针在两段里各自被扣掉, 因为它们是"插入"而非"移动". 结论一句话: **T 是缝到缝 (lo, hi) , $sweep$ 是位置到位置 $[L, R]$, 停 R 落 R 缝, 停 L 落 $L-1$ 缝**, 全部由"边界 = 右侧缝"一条规约推出.

J.cpp

```
// @Author : GoryK
#include <bits/stdc++.h>
using namespace std;

#define int int64_t

#ifdef LOCAL
#include "__DEBUG_TOOL.h"
#else
#define debug(...)
#endif

// REGISTER_REFLECT()

class SGT {
```

```
std::vector<int> seg;
int off = 0;
void pull(int i) { seg[i] = seg[i << 1] + seg[i << 1 | 1]; }
public:
SGT() = delete;
SGT(int n) {
    off = std::bit_ceil(unsigned(n + 2));
    seg.assign(off << 1, 0);
}
void add(int i) {
    seg[i += off]++;
    while (i >= 1) pull(i);
}
int ask(int l, int r) {
    if (l > r) return 0;
    int res = 0;
    for (l += off - 1, r += off + 1; l ^ r ^ 1; l >>= 1, r >>= 1) {
        if (~l & 1) res += seg[l ^ 1];
        if (r & 1) res += seg[r ^ 1];
    }
    return res;
}
};

void solve() {
    int n; cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    map<int, int> L, R;
    map<int, vector<int>> pos;
    for (int i = 1; i <= n; i++) {
        R[a[i]] = i;
        pos[a[i]].push_back(i);
    }

    for (int i = n; i >= 1; i--) {
        L[a[i]] = i;
    }

    auto v = a;
    sort(v.rbegin(), v.rend() - 1);
    v.erase(unique(v.begin(), v.end()), v.end());
    // debug(v);

    SGT seg(n + 1);

    L[0] = 1; R[0] = 1;
```

```

// 区间内 已经被删掉的元素数量
auto del = [&](int l, int r) → int { return seg.ask(l, r); };

vector<array<int, 2>> dp(v.size(), {INT_MAX, INT_MAX});
dp[0] = {0, 0};

for (int i = 1; i < (int)dp.size(); i++) {
    auto& P = pos[v[i]];
    int cl = L[v[i]], cr = R[v[i]], k = (int)P.size();

    // lst_l → L → R
    // lst_l → R → L
    // lst_r → L → R
    // lst_r → R → L

    // 这个区间有多少这次要删除的元素
    auto pc = [&](int l, int r) → int {
        // [l, r)
        return (int)(upper_bound(P.begin(), P.end(), r)
            - lower_bound(P.begin(), P.end(), l));
    };

    auto T = [&](int b1, int b2) → int {
        int lo = min(b1, b2), hi = max(b1, b2);
        if (lo == hi) return 0;
        return (hi - lo) - del(lo + 1, hi) - pc(lo + 1, hi);
    };

    // 除掉 之前的删掉的数字, 和这次要删掉的数字, 从 cl 到 cr 还剩下多少数字
    int sweep = (cr - cl + 1) - del(cl, cr) - k;
    int pr = (i == 1) ? 0 : R[v[i - 1]];
    int pl = (i == 1) ? 0 : L[v[i - 1]] - 1;

    // lst_d → cur_d ^ 1 → cur_d
    // 等于 走完 [L, R] 的完整的没删的 + lst_d → cur_d ^ 1 的删掉的
    // 也就是 T(lst_d, cur_d ^ 1) + sweep

    // 用每个边界的左边的数字代替这个边界
    // 假如边界两段都是要删除的元素, 操作就是从 r 的右端移动到 l 的右端, 再加上删掉
    // l 的操作

    dp[i][0] = min(dp[i - 1][0] + T(pr, cl - 1), dp[i - 1][1] + T(pl, cl - 1)) + sweep;
    dp[i][1] = min(dp[i - 1][0] + T(pr, cr), dp[i - 1][1] + T(pl, cr)) + sweep;

    for (int& p : P) seg.add(p);
}

```

```
    cout << n + min(dp.back()[0], dp.back()[1]) << "\n";
}

int32_t main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    int tt = 1, _ = 0;
    std::cin >> tt;
    while (tt-- and ++_)
        // std::cout << "TEST CASE : " << _ << endl,
        solve();

    return 0;
}
```

Problem K. 最小生成树

K. 最小生成树

Kruskal

贪心

分类讨论

中

复杂度 $O(m \log m)$

k 个特殊点在生成树中必须是叶子, 即特殊点之间不能相邻 ($n > 2$ 时). 把边分成两类: 两端都是普通点的 $edges_1$, 至少一端是特殊点的 $edges_2$.

两阶段 Kruskal: 先用 $edges_1$ 把 $n-k$ 个普通点连成一棵树 (连不成输出 -1); 再对 $edges_2$ 排序跑 Kruskal, $n > 2$ 时跳过两端都是特殊点的边, 把每个特殊点挂到普通点骨架上. 正确性: 特殊点是叶子意味着删掉它们后剩余普通点必须自身连通, 两阶段各自取最小恰好是全局最优.

注意 $n = 2$ 的特判: 此时允许两个特殊点直接相连.

K.cpp

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
#define ll long long

class DSU {
public:
    vector<int> f, siz, dis;
    DSU(int n) {
        f.resize(n);
        iota(f.begin(), f.end(), 0LL);
        siz.assign(n, 1);
        dis.assign(n, 0);
    }

    int find(int x) {
        if (x != f[x]) {
            int ffx = find(f[x]);
            f[x] = ffx;
        }
        return f[x];
    }

    bool same(int x, int y) {
        return find(x) == find(y);
    }
}
```



```
bool merge(int x, int y) {
    int fx = find(x), fy = find(y);
    if (fx == fy) {
        return false;
    }
    f[fx] = fy;
    siz[fy] += siz[fx];
    return true;
}

int size(int x) {
    return siz[find(x)];
}
};

void solve() {
    int n, m, k;
    cin >> n >> m >> k;

    map<int, int> spe;
    for (int i = 1; i ≤ k; i++) {
        int v; cin >> v;
        spe[v] = 1;
    }

    vector<array<int, 3>> edges;
    for (int i = 1; i ≤ m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        edges.push_back({u, v, w});
    }

    vector<array<int, 3>> edges1, edges2;
    for (const auto&[u, v, w] : edges) {
        if (spe[u] or spe[v]) {
            edges2.push_back({u, v, w});
        } else {
            edges1.push_back({u, v, w});
        }
    }

    sort(edges1.begin(), edges1.end(), [&](const auto &a, const auto &b){
        return a[2] < b[2];
    });

    sort(edges2.begin(), edges2.end(), [&](const auto &a, const auto &b){
        return a[2] < b[2];
    });

    DSU dsu(n + 1);
```

```
int ans = 0, cnt = 1;
for (auto &[u, v, w] : edges1) {
    if (dsu.same(u, v)) {
        continue;
    } else {
        dsu.merge(u, v);
        ans += w;
        cnt++;
    }
    if (cnt == n - k) break;
}

if (cnt < n - k) {
    cout << -1 << '\n';
    return;
}

for (auto &[u, v, w] : edges2) {
    if (n > 2 and spe[u] + spe[v] > 1 or dsu.same(u, v)) {
        continue;
    } else {
        dsu.merge(u, v);
        ans += w;
        cnt++;
    }
    if (cnt == n) break;
}

if (cnt < n) {
    cout << -1 << "\n";
    return;
}

cout << ans << "\n";
}

int32_t main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

    int tt = 1;
    cin >> tt;
    while (tt --) {
        solve();
    }

    return 0;
}
```

Problem L. 分数迭代

L. 分数迭代

数论

gcd

断点预处理

难

复杂度 $O(\sqrt{V} + \Omega(\text{diff}) + q \log)$

化简分数 $(a+k)/(b+k)$. 关键恒等式: $\gcd(a+k, b+k) = \gcd(a+k, |a-b|)$, 所以公因子只可能来自 $\text{diff} = |a-b|$ 的质因子, 且随 k 变化的"约分结构"只有 $O(\Omega(\text{diff})) \approx 40$ 个断点.

预处理: 线性筛 + 分解 diff ; 从 $k = 0$ 出发, 每轮在剩余质因子中找最小的增量 k , 使当前约简后的 (a', b') 能再吃掉一个质因子, 记录断点 $\rightarrow$ 约简结果的映射 (map).

查询: upper_bound 定位到 $\leq k$ 的最后一个断点, 在该断点的约简结果上平移增量 d , 最后再做一次 gcd 兜底化简输出.

L.cpp

```
// @Author : GoryK
#include <bits/stdc++.h>
using namespace std;

#define int int64_t

#ifdef LOCAL
#include "__DEBUG_TOOL.h"
#else
#define debug(...)
#endif

// REGISTER_REFLECT()

const int N = 1e6;
vector<int> pr;
vector<int> vis(N + 1, 0);

void prime(int n) {
    vis[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!vis[i]) pr.push_back(i);
        for (const auto& j : pr) {
            if (i * j > n) break;
            vis[i * j] = 1;
            if (i % j == 0) break;
        }
    }
}
```

```
map<int, int> cal_fac(int n) {
    map<int, int> res;
    for (int& p : pr) {
        if (p * p > n) break;
        while (n % p == 0) { res[p]++; n /= p; }
    }
    if (n > 1) res[n] ++;
    return res;
}

void solve() {
    int a, b, q;
    cin >> a >> b >> q;
    int diff = abs(a - b);
    auto fac = cal_fac(diff);

    auto cal = [&](int a, int i) -> int {
        return (i - a % i) % i;
    };

    // 问题转化成, 对于一个  $\gcd(a, b) = 1$  来说, 如何快速找到
    //  $\gcd(a + k, b + k) = i$  的最小  $k$ 
    using pii = pair<int, int>;
    map<int, pii> ans;
    ans[0] = pair(a, b);
    while (fac.size()) {
        int g = 1, k = LLONG_MAX;
        auto [pre, TEMP] = *prev(ans.end());
        auto [cur_a, cur_b] = TEMP;
        // 找最小的  $k$ , 且有  $\gcd(a + k, b + k) = g$ 
        for (auto &[f, cnt] : fac) {
            int res = cal(cur_a, f);
            if (res < k) {
                k = res;
                g = f;
            }
        }

        ans[pre + k] = pair((cur_a + k) / g, (cur_b + k) / g);
        if (!--fac[g]) fac.erase(g);
    }

    // debug(ans);

    for (int i = 1; i <= q; i++) {
        int k; cin >> k;
        auto it = prev(ans.upper_bound(k));
        int d = k - it -> first;
        int A = it -> second.first + d,
```

```
        B = it → second.second + d,
        G = __gcd(A, B);
        cout << A / G << ' ' << B / G << "\n";
    }
}

int32_t main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    std::cout.tie(nullptr);

    prime(N);

    int tt = 1, _ = 0;
    std::cin >> tt;
    while (tt-- and ++_)
        // std::cout << "TEST CASE : " << _ << endl,
        solve();

    return 0;
}
```

7.15 补题报告 · 2026 SDCPC · 25 李增睿 · One Dark / macOS style